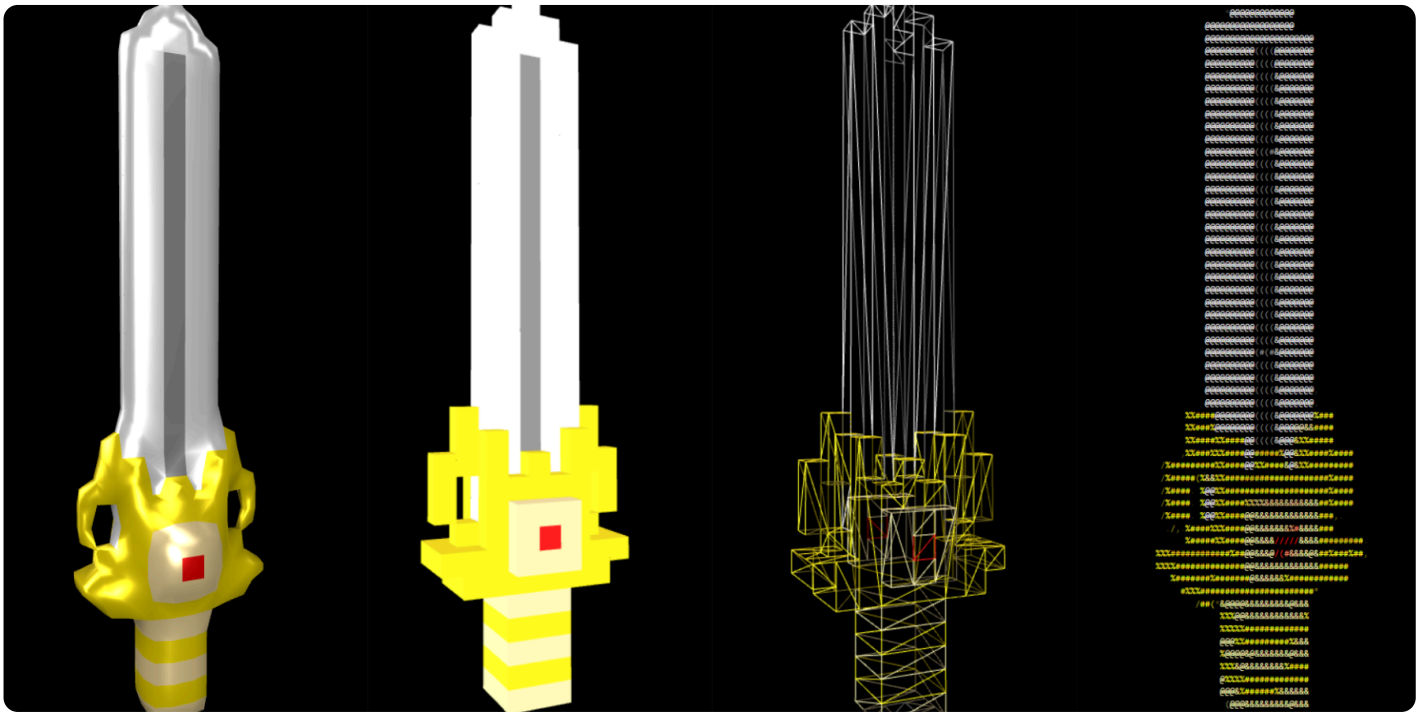


Immortalizing Wearables

How to make a snapshot of all the assets and metadata from the original Cryptovoxels wearables collection live onchain forever

In this post we propose a solution to improve the metadata of Cryptovoxels wearables by combining it with glTF 3D models, then explore some of the possibilities for hosting metaverse assets directly on-chain. We took a snapshot on 11-1-2022 of all the assets and metadata from the original Cryptovoxels wearables collection, link below:

📂 **GitHub: https://github.com/m3-org/cryptovoxels-wearables** ⭐

There are over 3000+ voxel wearables made by hundreds of artists in the openvoxels dataset which represents a fraction of how many there are total. Today there are 847 collections and **33695** minted collectibles from **1142** authors on the Voxels website.

We hope this post can provide information that's useful to those working on avatar wearables and/or working groups focused on interoperable open metaverse standards.

> Note: If interested in topics like archiving metaverse assets, crypto, AI, XR, come drop by the M3 discord and make an introduction 😊

Upgrading Wearables Metadata

Cryptovoxels Wearables are one of the earliest examples of 3D NFTs (Chainbreakers comes in first) that point to the hash / file location of the actual model in the metadata. However, the metadata of the collection is currently centralized which is always a risk for long term historical preservation and collector value.

Here's a proposal on how we can improve the metadata by uploading assets to IPFS / arweave and updating the fields, including adding new fields for file formats such as glb and svox:



Current metadata for the first wearable on the left vs proposed spec on the right (IPFS/arweave + glb/svox formats)

We are also looking into using Filecoin to archive this dataset. The release of FVM (Filecoin Virtual Machine) is eminent which makes the timing prescient.

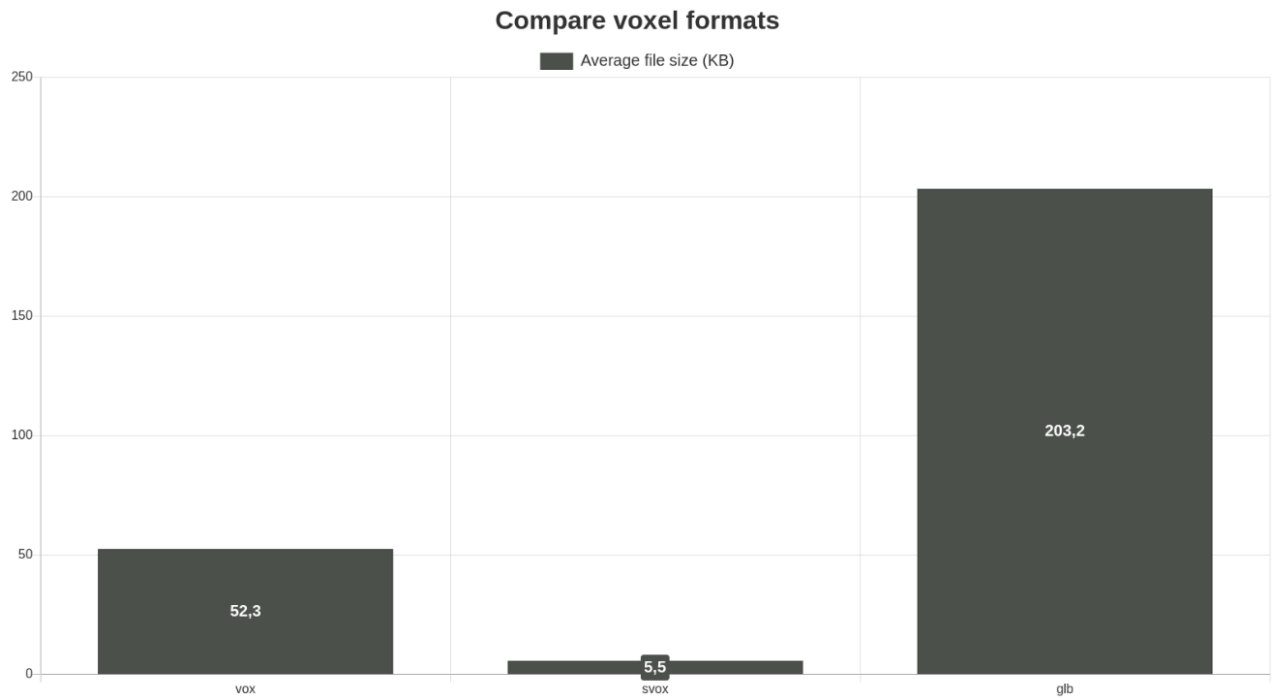
File Formats

3D pixels aka voxels have many different file types and implementations. Voxels uses the non-standard file format .vox derived from the popular editor MagicaVoxel for in-game assets and wearables. We converted all of the files in the dataset to svox (the markdown of voxel formats) and glTF 2.0 binary files that each offer unique benefits:

- glTF 2.0 is released as an ISO/IEC International Standard
    - Open standard file formats better for longevity
- svox (smooth voxels) has the smallest footprint + human readability

Here are some stats on the average file size differences WITHOUT compression:

```
# Calculate average size of files in directory
ls -l | awk '{s+=$5} END {print "Average file size: " s/NR/1024"k"}'
```



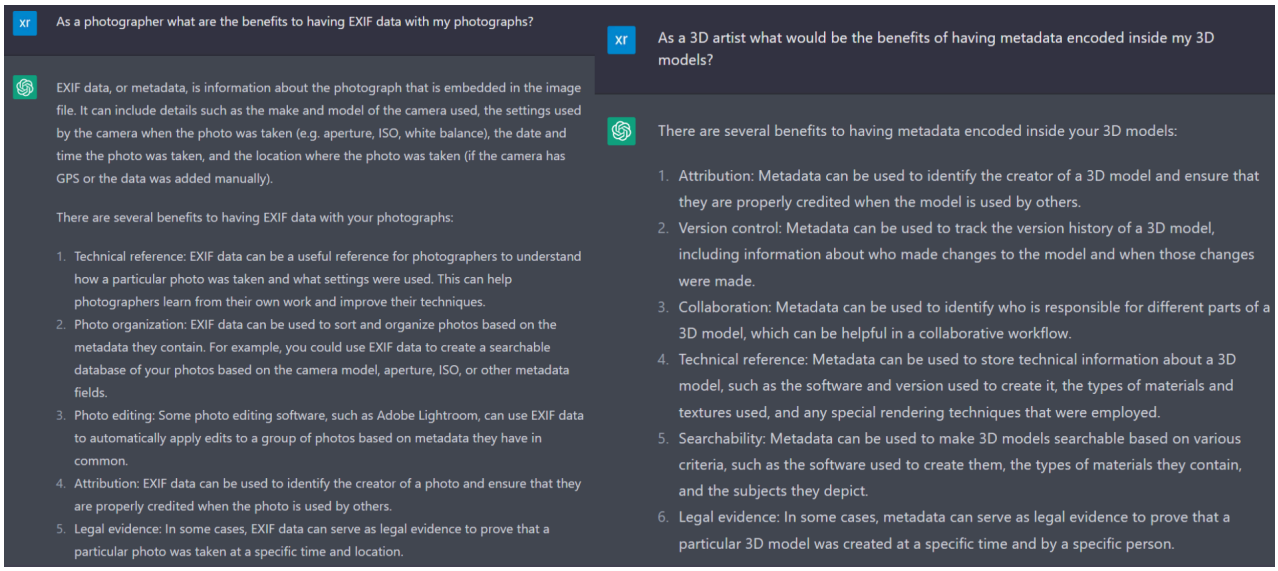
Average file size of all 3166 wearables with no glTF extensions or compression applied

There’s an option in [svox-tools] to compress when converting from vox2svox which brings down the average file size for svox down to ~**2 KB** which is small and cost effective enough to host fully on-chain! More on this soon…

Combining Metadata with 3D Model

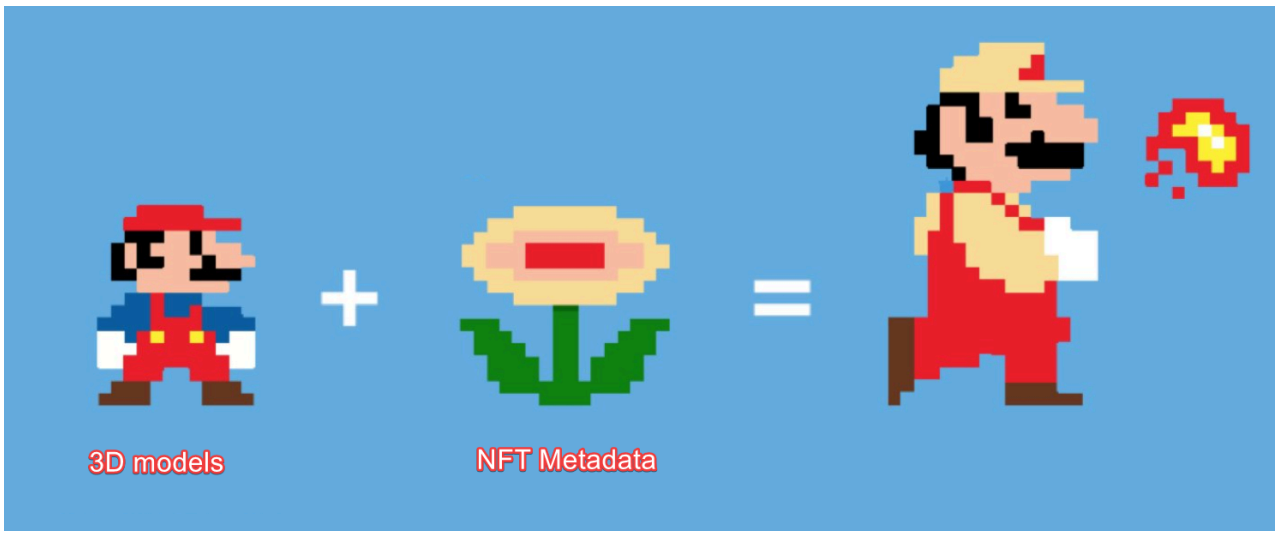
The current wearables collection uses MagicaVoxel .vox format (as mentioned above) which isn’t an open standard file format nor does it have a means of holding metadata within the file itself.

Benefits of combining metadata with digital art



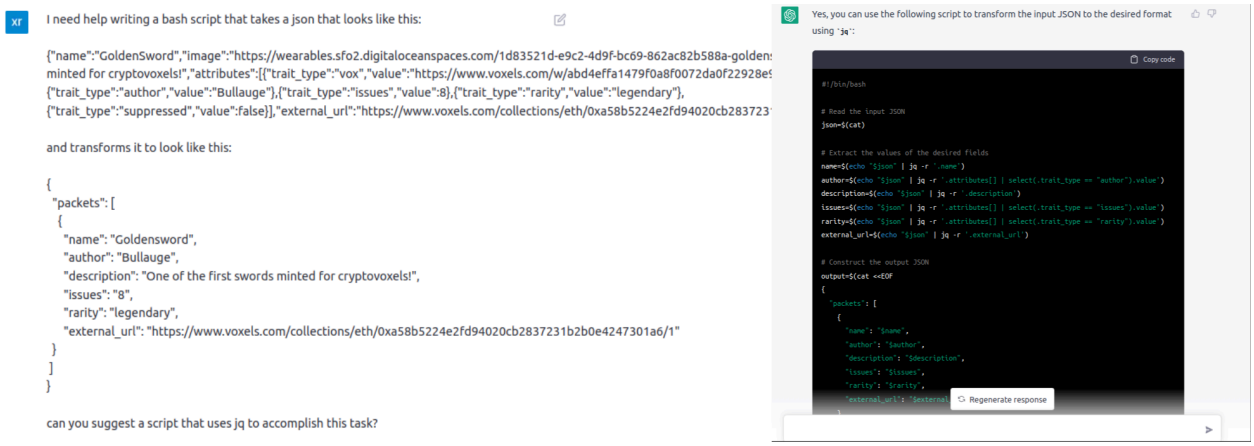
Many of these benefits, especially attribution, are similar with how crypto enthusiasts discuss NFTs

TL;DR



Combining 3D models with the metadata is better for interoperability than having them separate

Converting metadata from one JSON schema to another



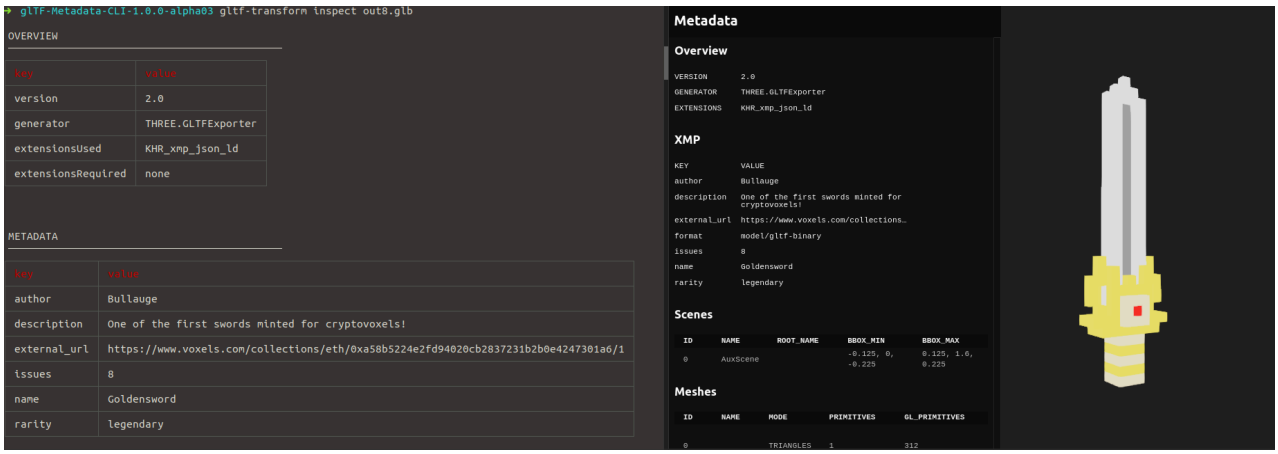
ChatGPT is really good at transforming data from one format or spec to another

See the finished script here: https://github.com/M3-org/cryptovoxels-wearables/blob/main/scripts/convert_json.sh

Combine JSON with glTF files

I used this tool from KhronosGroup to programmatically add JSON to glTF: https://github.com/KhronosGroup/glTF-Metadata-CLI

example command: `cargo run -- -i 1.glb -o out8.glb -j 1_new.json`



Verifying with gltf-transform and gltf.report that metadata was added to the glTF model

---

On-chain Wearables

✋ Everything from here on are notes for future exploration 📝

There was an epic thread from @cybourgeoisie breaking down how CyberBrokers art was uploaded on-chain:

https://twitter.com/cybourgeoisie/status/1503130088740638725

> *Openvoxels doesn't have enough ETH to upload the entire voxel wearables collection to the Ethereum blockchain chain incase you're wondering*

The on-chain art crowd is where some of the dopest builders in web3 are. On September 13th dom shared a post of an on-chain threejs app of a rose:

https://twitter.com/dhof/status/1569775925679558660

Threejs is the most popular Javascript library for 3D graphics on web and powers a lot of the virtual world platforms many of us are building. This means artists can create virtual worlds stored as static HTML that reference on-chain 3D graphics libraries, making for a way more decentralized and censorship-proof medium of expression.

I asked in mathcastles discord how much it might cost to store 1kb on-chain, here's a function 113 mentioned that offers a roughly accurate estimate:

```
function ethCost(kb, gweiPrice){

    const SSTORE2 = 245522
    const SSTORE = 640000;

    const writeStyle = SSTORE2

    return (kb * writeStyle * gweiPrice) /1000000000
}
```

> 113: iirc with sstore2 it varies a little based on how big the total blob you're putting up is. at current 20 gwei gas price you'd pay ~.005 eth per kb
>
> xaltgeist: the breakpoint is around 32 bytes, at which point SSTORE2 is more efficient. so if you're storing any appreciable amount of data in a given transaction, SSTORE2 is likely cheaper

Ethereum File System

EthFS (Ethereum File System) is an open source project that allows you to browse and upload on-chain files.

What?

dhof's on-chain 3D rose NFT inspired tons of fun conversations and projects. It surfaced many examples and use cases for highly reusable on-chain assets (JS libraries, fonts, gifs).

EthFS is an effort to consolidate these assets into a public, durable on-chain repository with ergonomic, gas-optimized methods for reading and writing files.

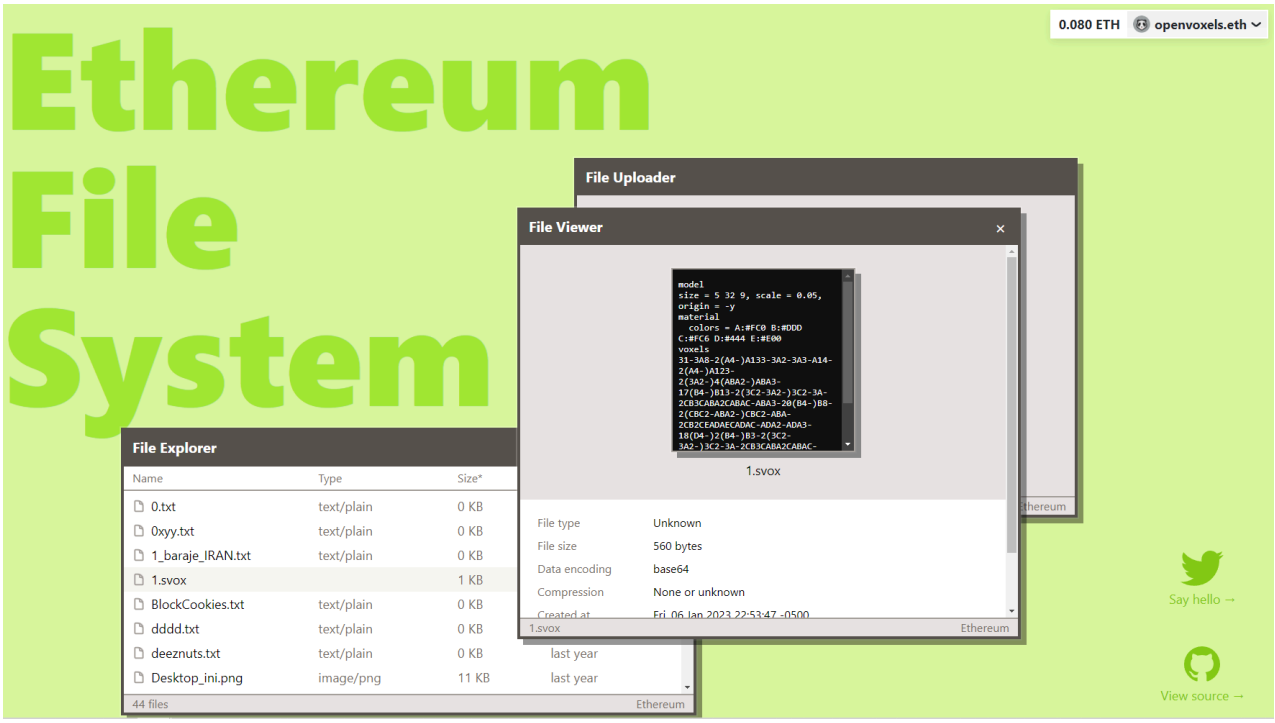
How does it work?

> EthFS leans on SSTORE2's strategy to efficiently read and write data in the form of contract bytecode. The frontend splits files into 24kb chunks (max contract size) and writes them to the `ContentStore`, a content-addressable data storage contract.
>
> A `File` struct wraps a set of content checksums that, when concatenated in order, represent the full file contents. It also holds the logic for reading/unwrapping the file contents into a single string in a very gas-efficient way.
>
> The `FileStore` contract acts as a minimum viable registry, a global namespace of human-readable filenames for more ergonomic use within consumer contracts and the frontend's file browser.
>
> > For specific use cases (e.g. different naming schemes, permissioning, etc.), you may decide to create your own file store abstraction. That's great! The `ContentStore` is designed to be unopinionated (strictly content-addressable), so that building on top of it can benefit everyone.

https://ethfs.xyz/

Some napkin math on how much uploading all the svox files on-chain would cost:

- 1 average file = 2 KB → 3 KB base64-encoded - costs 2 transactions (one per 24 KB chunk, one for file metadata) - estimated ~0.022 ETH in total gas fees - 0.022 * 3138 files = 69 ETH *nice*



It cost 0.0054 ETH or about tree-fiddy

https://etherscan.io/tx/0x863c09df954472af6b8971ef24bfa3a328363d8cebfc95789d8fadd4e9353d9f transaction with the base64 encoded 1.svox file (Goldensword)

On-chain Worlds

More breadcrumbs to follow..

https://twitter.com/xtremetom/status/1600542212735090711

The steps go something like this:

1. All scripts are minified and bundled
2. Convert bundle to PNG (DEFLATE)
3. Compress PNG
4. Convert PNG to base64
5. Split data into 24kb chunks
6. Store with SSTORE2 (LibraryStorage.sol)
7. (INFLATE) Pass the PNG to WebGL and convert RBGA back to bundle

---

Honorary mention

No matter if blockchain, IPFS, or Arweave, there's no feeling of ownership that hits as hard as having a cold copy of your data.

This post was **not** sponsored by them, I just wanted to give a quick shoutout to ClubNFT, they built a free tool that you can use to backup the actual files for all of your NFTs and safely secure them in cold storage (such as a backup drive / USB) 👏

